ITDMA3-34 · Final Presentation and System Demonstration

MMM Booking App

An Android booking system for MMM Social Workers and Associates, Gauteng

Group 11 · Faith Murau · Khauhelo Moqechane · Ignatius Mayibongwe Khumalo · Eduvos

Presenter: All three

Our team and roles

Faith Murau

UX/UI Design and Testing

Look and feel, messages, UX test pack (24 tests)

Khauhelo Moqechane

Frontend Development, Screen and Navigation Testing

22 screens, navigation, navigation test pack (18 tests)

Ignatius Mayibongwe Khumalo

Backend, Database, Authentication, Booking Logic Testing, GitHub and Final Submission

Firebase, rules, 61 backend tests

Shared by all three: integration testing, demo video, final report.

Presenter: All three

The problem and our purpose

The problem

A psychology and social work practice across Midrand, Johannesburg and Centurion, plus online sessions, needs to manage appointments without clashes, gaps or admin back-and-forth.

For clients

Book in minutes, see only free times, get reminders, cancel within clear rules.

For the practice

One place to confirm, decline, track payment, set breaks and leave, and see analytics.

Presenter: Faith

Executive summary

Goals

Reliable, conflict-free online booking for clients and admins.

Scope

Android app, 22 screens, two roles, Firebase services, no custom server.

Timeline

Started 20 July 2026. Six deliverables, last one due 13 November 2026. Oral presentation 25 November.

Outcomes

All 282 planned tests passed. Real phone and tablet tested. Runs on the free Firebase plan.

Business value: fewer double bookings, less admin time, a clear cancellation policy, data kept in Johannesburg.

Presenter: Faith

Our roles 1 of 3 · Faith Murau

UX/UI design and testing

Design

Dark green and beige palette, light and dark themes, card layouts, 48dp touch targets, tablet-friendly sizes.

Clear messages

Inline field errors, check-your-email dialog, What happens next card after booking.

UX testing

24 tests (UX-01 to UX-24) on consistency, clarity, error and confirmation messages. All passed.

Presenter: Faith

Testing and validation 2 of 2

User acceptance, feedback and data

User acceptance

46 manual functional, usability and edge tests on a real phone and tablet. 24 UX and 18 navigation tests.

Feedback acted on

Supervisor raised 5 points: email check, terms, cancellation, availability, clarity. All fixed and verified by 23 tests.

Data and evidence

Simulated bookings and accounts. 27 defects logged and fixed. Screenshot evidence for every test ID.

Result: 282 of 282 tests passed.

Presenter: Faith

Challenges and resolutions · Faith

UX and UI challenges

Challenge	How we solved it	Where in the code
Registration did not tell users to check email	Check-your-email dialog names the address and mentions Spam. If sending fails it says so and points to Resend.	auth/RegisterActivity.sendVerificationEmail(), strings.xml
Supervisor: UI not obvious	Booking confirmation now shows the clinic address, a What happens next card and clear buttons.	client/BookingConfirmationActivity, activity_booking_confirmation.xml
Terms of use too thin	Rewritten into nine sections including cancellation, no-shows and fees.	strings.xml (terms_body), settings/TermsActivity
Small icon buttons hard to tap	Expanded to the 48dp touch-target guideline.	utils/TouchTargetUtils.java
Dark mode readability	Full dark palette and saved theme preference.	res/values-night/, utils/ThemeManager.java, MMMApplication

Presenter: Faith

Requirements coverage · Faith

UX and documentation requirements

Requirement	Status	Implementation and files
Responsive UI	100%	Phone and tablet, portrait and landscape: MaxWidthLinearLayout, values/dimens.xml, values-sw600dp/dimens.xml
Accessibility	100%	contentDescription on icons, TouchTargetUtils (48dp), Dark Mode palette in values-night
Error Handling	100%	Inline field errors, offline guards, clear slot-taken, closed-day and holiday messages
User Guide	100%	USER_GUIDE.md for client and admin, with troubleshooting table
Documentation	100%	README.md, USER_GUIDE.md, TESTING.md, firestore.rules.md, code comments
Innovation Feature	100%	Offline mode, role-based access, analytics dashboard, labelled trend projection

Presenter: Faith

Our roles 2 of 3 · Khauhelo Moqechane

Frontend development and navigation

22 screens

Auth, client, settings and admin screens in Java with XML layouts and RecyclerView adapters.

Navigation and responsive

Back-arrow rule, double-back to exit, phone and tablet layouts with width-capped content.

Screen testing

18 tests (NAV-01 to NAV-18) on the screen inventory and navigation flow. All passed.

Presenter: Khauhelo

Project structure

```
MMM-Booking-App/
├── README.md, USER_GUIDE.md, TESTING.md
├── firestore.rules.md (deployed rules)
├── Testing_Evidence ... .pdf, demo .mp4
├── build.gradle.kts, settings.gradle.kts
└── app/
    ├── google-services.json
    └── src/
        ├── main/
        │   ├── AndroidManifest.xml
        │   ├── java/com/mmm/bookingapp/
        │   └── res/ layout (27), values,
        │       values-night, values-sw600dp,
        │       drawable, xml, mipmap
        ├── test/ 81 JUnit tests (5 files)
        └── androidTest/ 1 Espresso test
```

```
java/com/mmm/bookingapp/
├── MainActivity, MMMApplication
├── auth/       Welcome, Register,
│               Login, ForgotPassword
├── client/     Home, Services, Book,
│               Confirmation, MyBookings,
│               Contact
├── settings/   Settings, Password, Email,
│               About, Privacy, Terms, FAQ
├── admin/      Dashboard, BookingDetail,
│               PracticeInfo, Analytics
├── models/     User, Booking,
│               BookedSlot, PracticeInfo
├── adapters/   Booking, Service, FAQ
└── utils/      16 helpers: validation,
                booking actions, availability,
                holiday API, notifications,
                CSV export, theme
```

Presenter: Khauhelo

Core feature 1 of 3

Smart, conflict-free booking

What it does

Client picks service, type, location, date and length. Only free start times are offered.

Rules enforced

Open days, hours, breaks, leave, South African public holidays (live API), quarter-hour starts.

Advanced interaction

One slot document per 15 minutes, written in a single transaction, so two people can never book the same time.

Demo: book at a free time, try a holiday, try a taken slot.

Presenter: Khauhelo

Booking by location

Midrand, Johannesburg and Centurion

Midrand

Life Carstenhof Clinic, 21 Dane Rd, Glen Austin AH, 1685

Johannesburg

19 Ameshoff Street, Braamfontein, 2001

Centurion

162 Panorama Road, Rooihuiskraal, 0154

Each location has its own calendar: slot IDs include the location, so the same time can be booked at two clinics. Online sessions need no location. Address shown on booking: strings.xml (location_addresses), BookAppointmentActivity.showAddressFor(). Slots: BookedSlot.buildId().

Presenter: Khauhelo

Core feature 3 of 3

Admin management and analytics

Booking control

Confirm, decline, mark paid or completed, delete. Cancel and decline free the slot atomically.

Practice settings

Open days, hours, daily breaks, blocked dates and leave with reasons.

Insight and export

Analytics using server-side counts, and CSV export of filtered bookings.

Secured by roles: admins cannot be created from the app, and clients cannot change their role.

Presenter: Khauhelo

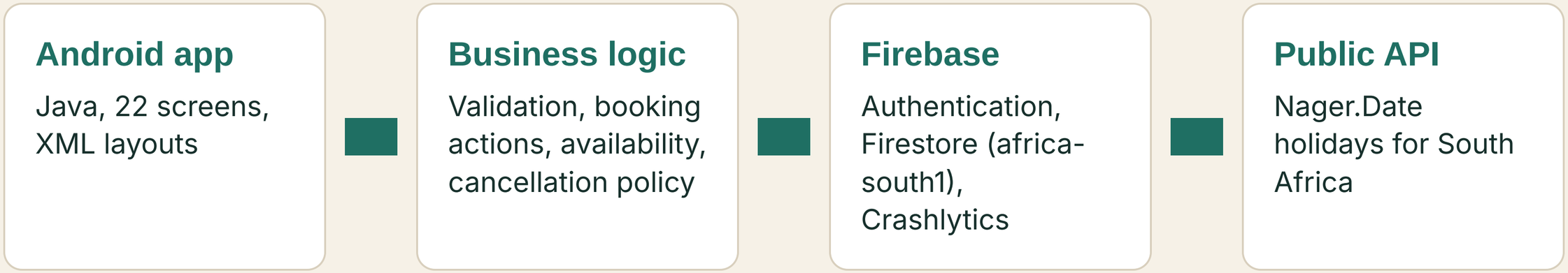
Frontend challenges

Challenge	How we solved it	Where in the code
Content stretched edge to edge on tablets	Width-capped, centred layout and larger text on screens 600dp and wider.	utils/MaxWidthLinearLayout.java, res/values-sw600dp/dimens.xml
Short forms stuck at top of tall screens	Vertical centring with fillViewport so landscape still scrolls.	activity_welcome.xml, activity_login.xml, activity_register.xml
Clients saw times that were already taken	Time picker lists only free times and clears when date, length or location changes.	utils/AvailabilityUtils.freeStartTimes(), BookAppointmentActivity.showFreeTimes()
Clients unsure where to go for in-person	Selected clinic address appears under the location dropdown, hidden for online.	strings.xml (location_addresses), BookAppointmentActivity.showAddressFor()
Back button could sign users out	Root screens use double-press to exit; login clears the back stack.	HomeActivity, AdminDashboardActivity, FLAG_ACTIVITY_CLEAR_TASK
Admin list renders all rows	Accepted trade-off at one practice; fix is one RecyclerView with a header.	admin/AdminDashboardActivity, activity_admin_dashboard.xml

Frontend requirements

Requirement	Status	Implementation and files
Navigation	100%	22 screens, back arrow except Welcome, Home, Admin Dashboard, double-back exit, stack cleared on login
Search and Filtering	100%	AdminDashboardActivity.applyFilters(), matchesSearch(): text search plus status, location, date
Data Visualisation	100%	Status bar updateAnalyticsBars(), AdminAnalyticsActivity.updateChartBars(), no chart library
Input Validation	100%	ValidationUtils.java, PracticeInfoActivity hours checks, booking rules, 81 unit tests
Notifications	90%	Local confirmation and evening-before reminder, NotificationHelper.java. Admin-to-client push needs paid plan
Performance	100%	count() queries, live listeners, vector icons, RecyclerView
Offline Functionality	100%	NetworkStatusHelper.java, SessionCache.java, Firestore cache, offline banners

System architecture



Serverless: the app talks to Firebase directly. Security Rules enforce access on the server side.

Presenter: Ignatius

Technology choices

Layer	Technology	Why
Frontend	Java, Android (API 24+), XML	Native speed, tablet support via resource qualifiers
Backend	Firebase Auth, Security Rules	Managed, secure, no server to maintain
Database	Cloud Firestore (NoSQL)	Real-time listeners and transactions
Third party	Nager.Date API, Crashlytics	Holiday blocking, crash monitoring
Testing	JUnit 4, Espresso	Automated logic and UI checks

Presenter: Ignatius

Our roles 3 of 3 · Ignatius Mayibongwe Khumalo

Backend, database and authentication

Backend

Firebase Authentication with email verification, four Firestore collections, transactions, live listeners, Security Rules.

Booking logic

Open days, holidays, breaks, 24-hour rule, conflict-free slots. GitHub repository and final submission.

Backend testing

61 tests: database 21, booking logic 22, permissions 18. All passed.

Together: 12 two-phone integration tests, demo video, final report.

Presenter: Ignatius

Core feature 2 of 3

Real-time sync between app and web services

Client books
Booking saved in Firestore

Admin sees it
Dashboard and counts update live

Admin confirms
Client's status badge changes, no refresh

Firestore snapshot listeners, Firebase Authentication login, local confirmation and evening-before reminders, Nager.Date holiday API.

Presenter: Ignatius

Testing and validation 1 of 2

Automated, integration and backend tests

81
unit tests: validation, booking rules, cancellation, availability

12
integration tests with two phones

37
hardening checks on a real phone

18
permission tests in Rules Playground

Backend: 21 database and authentication tests, 22 booking logic tests. Security checked server-side: role cannot be changed, clients cannot read others' bookings, only admins delete. Espresso UI test on the Welcome screen.

Presenter: Ignatius

Backend challenges: data and booking

Challenge	How we solved it	Where in the code
Two clients booking the same time	Transactions cannot run queries, so each 15 minutes gets its own slot document, all read first and written together.	models/BookedSlot.buildBlockIds(), BookAppointmentActivity.submitBookingTransaction()
Cancelled booking could keep its time blocked	Cancel, decline and delete update the booking and release slots in one transaction.	utils/BookingActions.java
Same time at different clinics	Slot IDs include the location, so each clinic has its own calendar.	models/BookedSlot.buildId()
Auth account left behind if profile save failed	Account is deleted again (rollback).	auth/RegisterActivity.rollbackAuthAccount()
Holiday API unreachable	Booking continues on other rules and the client is told.	utils/HolidayApiClient.java, ensureHolidaysLoaded()

Backend challenges: security and plan limits

Challenge	How we solved it	Where in the code
Anyone could try to become admin	Rules allow new profiles only as client, role is never editable, admin checked server-side.	firestore.rules.md (users, bookings rules)
No push notifications on free plan	Local notifications and live listeners instead; limit documented.	utils/NotificationHelper.java, ReminderBroadcastReceiver.java
Cannot delete accounts without Cloud Functions	In-app request opens an email; admin deletes in Firebase Console.	settings/SettingsActivity (Delete My Account)
24-hour rule needed a policy	Free until 24 hours before, then contact the practice. Enforced in app, not rules.	utils/CancellationPolicy.java, client/MyBookingsActivity.onCancelClick()
Predictable analytics cost	count() aggregation instead of downloading documents.	admin/AdminAnalyticsActivity, utils/AnalyticsHelper.java

Requirements coverage · Ignatius 1 of 2

Backend requirements: access and data

Requirement	Status	Implementation and files
User Authentication	100%	Firebase email/password with verification: LoginActivity, RegisterActivity (rollback), ForgotPasswordActivity, MainActivity
Role-Based Access	100%	Server-side roles in firestore.rules.md. Routing: LoginActivity.routeByRole(), MainActivity.checkLoginStateAndRoute()
Database Integration	100%	Firestore: users, bookings, bookedSlots, practiceInfo. References in utils/FirebaseRefs.java
CRUD Operations	100%	Bookings full CRUD; user accounts create, read, update, delete via admin by design
API Integration	100%	Firebase SDK and Nager.Date holidays: HolidayApiClient.java, ensureHolidaysLoaded()
Security	100%	Deployed Security Rules block role self-assignment and slot tampering

Presenter: Ignatius

Requirements coverage · Ignatius 2 of 2

Backend requirements: operations and quality

Requirement	Status	Implementation and files
Real-Time Features	100%	addSnapshotListener in MyBookingsActivity and AdminDashboardActivity
Backup and Recovery	100%	Managed redundancy, Auto Backup rules, CSV export (CsvExportUtils.java), rules in Git
Logging and Monitoring	100%	Crashlytics and Log calls without personal data
Scalability	100%	Serverless Firestore, additive packages, centralised FirebaseRefs and BookingActions
Testing	100%	81 unit, 1 Espresso, 37 hardening, 46 manual, 115 team, 23 feedback: 282 of 282
Deployment	100%	Run on a real phone and tablet against live Firebase in africa-south1

Presenter: Ignatius

Live demonstration

Real-time operations

1. Register and log in
2. Book at a free time
3. Data saved in Firestore

App and web services

1. Second phone as admin confirms
2. Client screen updates live
3. Holiday API blocks a date

Unique interactions

1. Only free times shown
2. Late cancellation rule
3. Analytics and CSV export

Backup: recorded demo video from the repository.

Presenter: All three

Conclusions and recommendations

Delivered a working, tested booking system that met its goals.

Enhance

Push notifications via Cloud Functions, rescheduling, hours per weekday, fee calculation.

Maintain

Monitor Crashlytics, keep rules in version control, update dependencies, use CSV export as backup.

Scale

Firestore scales automatically. Move to Blaze plan, restructure the admin list for large volumes.

Presenter: All three

Questions and feedback

Why Firebase?

Managed, secure, real-time, free at this scale.

How is data handled?

Auth, server-side rules, no personal data in slot records.

Why Java?

Our first language, taught in class, and native to Android.

Design choices?

Own calendar per location, cancel and rebook, availability over strictness.

What would you want added first? Which screen felt least obvious?

Presenter: All three

Backup: development questions 1 of 2

Question	Answer
Why native Android in Java?	It was our first programming language, taught in class. It is native to Android, strongly typed, and one language covers all screens, logic and tests.
Why Firebase and no custom server?	Login, database, rules and live sync are managed, free on the Spark plan, and need no server upkeep.
Why Firestore?	Live listeners, transactions, automatic scaling, hosted in Johannesburg for low latency.
How do you stop double booking?	One slot document per 15 minutes, all checked and written in one transaction.
How did the team work together?	Roles split into UX, frontend and backend, one GitHub repository, shared integration tests.
What would you change?	Enforce the 24-hour rule server-side and add rescheduling.

Presenter: Ignatius

Backup: development questions

Why we built the whole app in Java

Our first language

Java was taught to us first, so all three of us could read, review and fix each other's code.

Native Android

Fast screens and full access to alarms, notifications, and tablet resources.

Works with Firebase

The official Android SDK supports Authentication, Firestore transactions and live listeners in Java.

Safer code

Strong typing catches mistakes when compiling. JUnit gave us 81 automated tests.

One language

22 screens, models, business logic and tests share one style, so the code stays consistent.

Mature tooling

Android Studio, large documentation and community, and long-term support from Google.

Presenter: Ignatius

Backup: development questions 2 of 2

Question	Answer
How is data protected?	Authentication plus server-side rules: owner-only reads, role cannot be changed. Slots hold no personal data and logs hold none.
What if the user is offline?	Cached data is readable. Login, booking and cancelling need a connection, with clear banners.
What if the holiday API fails?	Booking continues on the other rules and the client is told.
How was it tested?	282 tests on a real phone and tablet, with screenshot evidence.
How is an admin created?	Manually in the Firebase Console. The app has no admin sign-up.
What is not finished?	Server push, rescheduling, hours per weekday, fee calculation, instant account deletion.

Presenter: Ignatius

Backup: business questions 1 of 2 (proposals, not built)

How could the app make money?

Model	How it works	Needs
Save the practice money	Fewer no-shows through the 24-hour policy, less admin time, paid status tracked	Nothing new
Subscription	Other practices pay a monthly fee	Multi-practice support
Setup and support	Once-off setup plus monthly maintenance	Support agreement
Premium add-ons	Online payments, SMS or push reminders	Blaze plan, payment provider

Presenter: Faith

Backup: business questions 2 of 2

Costs, risks and who pays

Who pays

The practice. Clients use the app free.

Costs now

Firebase free Spark plan, Google Play registration once-off.

Costs later

Pay-as-you-go Blaze plan for push and growth.

Risks

Personal client data: privacy law (POPIA), support and updates.

Success measures: bookings created, late-cancellation rate, admin hours saved.

Presenter: Faith